# Lecture 1

# Introduction

Mingyu Gao

gaomy@tsinghua.edu.cn

*IIIS, Tsinghua*

# A Few Words about Myself

- Associate Professor in IIIS (since 2025.2.)
- Assistant Professor (2019.3. – 2025.1.)
- PhD from Stanford (2018); B.S. from Tsinghua (2012)
- Research: Computer Architecture
  - Domain-specific (e.g., AI) computing
  - Memory systems
  - Hardware security
  - …
- Contact: gaomy@tsinghua.edu.cn
- More on my webpage: http://people.iiis.tsinghua.edu.cn/~gaomy/

# About This Course

- An introduction of state-of-the-art computer systems
  - [What] To know basic system components
  - [Why] To understand key concepts and principles
  - [How] To learn practical skills and tools
  - [Do] To exercise actual application development
- Focus more on breadth than depth

- A gateway to more advanced system courses later
  - Computer architecture; operating systems; computer networking; database systems; distributed systems; parallel programming; …

# Course Logistics

- Lectures: Friday, 9:50am – 12:15pm, 六教6A017
  - Everyone is required to attend in classroom
- Website: 网络学堂 learn.tsinghua.edu.cn
  - Announcements, handouts, latest schedule, homework/lab submission, …
- Weekly review sessions (held by the TAs): time to be scheduled
  - Lecture review, homework & lab discussion, specific extended topics, …
- Make use of the WeChat group
  - Discuss common questions regarding homework, labs, lectures, …
  - Try not to private-message your TAs

# Course Logistics

- Instructors
  - Wei Xu 徐葳 (weixu@tsinghua.edu.cn, FIT 4-6004)
  - Mingyu Gao 高鸣宇 (gaomy@tsinghua.edu.cn, FIT 4-6015)
- TAs
  - Shuo Chen 陈硕 (s-chen25@mails.tsinghua.edu.cn)
  - Weitao Wan 万韦涛 (wwt22@mails.tsinghua.edu.cn)
  - Ruitao (Rita) Liu 刘睿韬 (liurt23@mails.tsinghua.edu.cn)
  - Tingrui Zhang 张庭瑞 (zhang-tr22@mails.tsinghua.edu.cn)






# Handouts and Textbooks

- Handouts available online before the class

- No required textbook; use handouts
- Books for reference
  - Randal E. Bryant and David R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, Third Edition, Pearson, 2016.
  - Abraham Silberschatz, Greg Gagne, Peter B. Galvin, *Operating System Concepts*, tenth Edition, Wiley, 2018.

# Course Load and Grading

- Midterm & final exams (25% + 25%)
  - Closed book, one page of double-sided A4 notes, calculators
- 3+1 homework assignments (15%)
  - Three will be graded
  - The last one is for self-practice and not graded
- 3 programming labs (30%)
  - Programming on real dev boards!
  - In teams of two
- Class participation (5%)
  - Using pop quizzes

# More on Labs

- Labs are the core of this course; system study needs to be *hands-on*!
  - Lab 1: performance optimization of a single-threaded program
  - Lab 2: make the program multi-threading with concurrent programming
  - Lab 3: extend the program into client-server over network

- Work in teams of 2 students
  - You should find your teammate as soon as possible!
  - Team members get the same points for each lab



- Each team will be provided a Raspberry Pi dev board
  - A widely used hardware platform for many cool designs
  - Like a complete computer, it has CPU, memory, networking, etc.
  - You need to return the device at the end of the course; take care of it!

# Programming Prerequisites

- You are expected to have basic knowledge about C/C++
- C is close to the underlying hardware
  - Allow us to accurately describe hardware features in high-level languages
  - Allow us to explicitly harness hardware features, e.g., parallelism, memory
  - Rust is similar; but Python, Perl, Java, Scala, Go, R, … are different

- Don’t panic; the first few lectures will help you review some key aspects
- This course is *NOT* about C/C++ programming
  - Check out the course 30470332, *Introduction to Programming in C/C++*

# Participation

- Class participation is EXTREMELY important!

- Ask questions
- Provide answers
  - Don’t worry about getting it wrong
- Teach us what you know well
  - And correct our mistakes on these topics
- Suggest topics
  - Let me know what you are interested in

# Rules

- Basic Honor Code
  - No plagiarism, cheating, … zero tolerance!
- Late policy
  - For every 24-hour period (rounded up) after deadline, take off 10% of total points
  - See Course Description doc for detailed explanation
- More in Course Description doc

# Computer Systems

# Systems From The Small to The Big

- Computer systems have diverse scales
- Two extremes: mobile & wearable ⇔ cloud & distributed systems







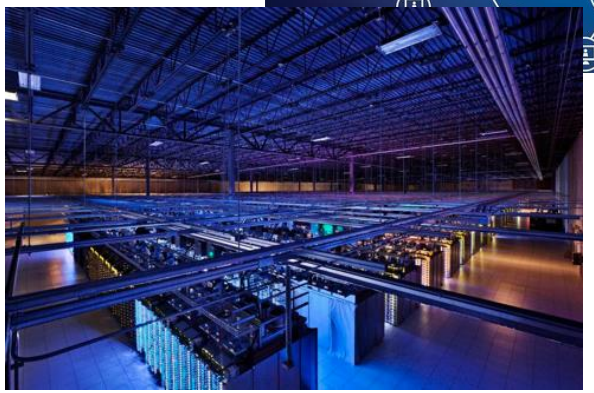




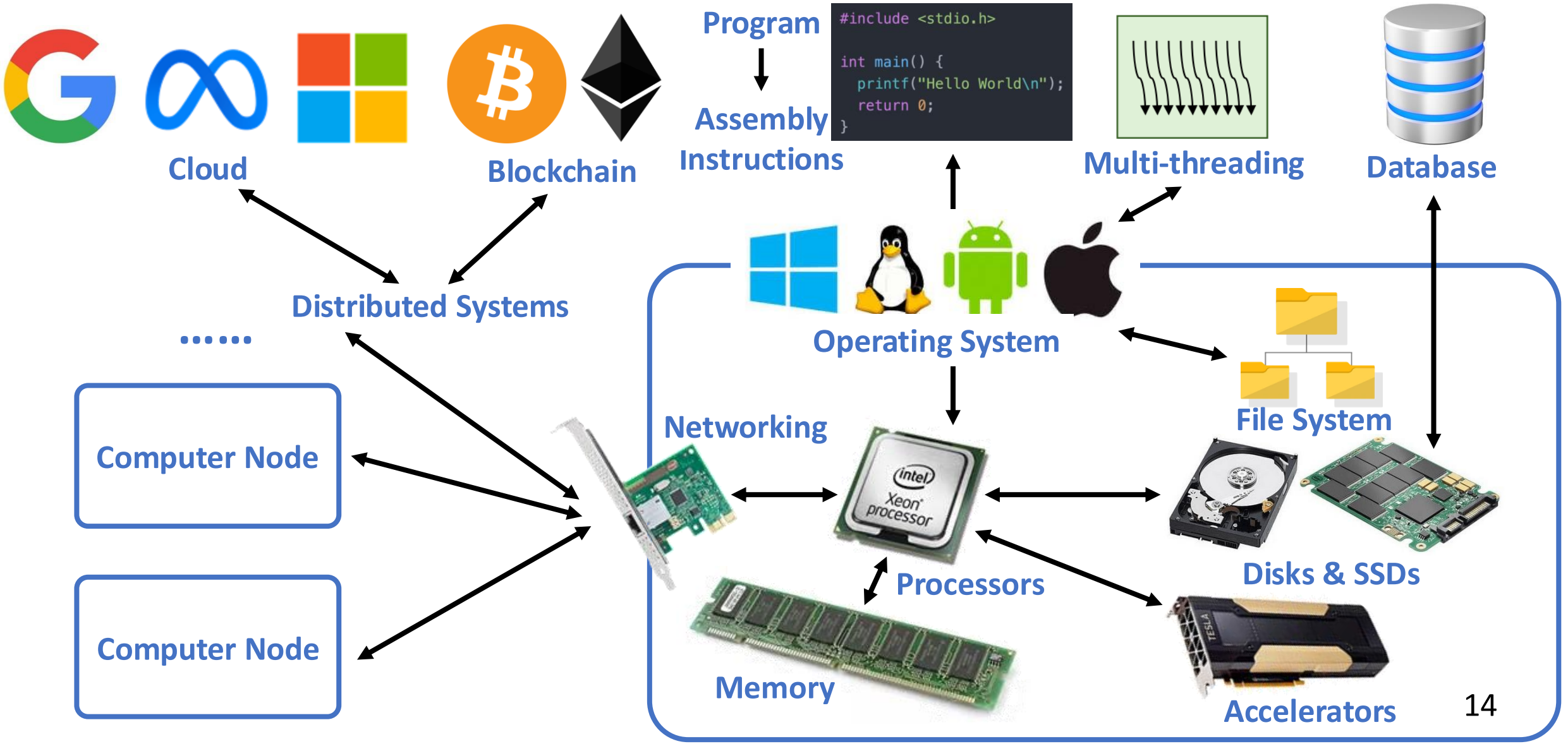








# The Scope of This Course



# Topics Covered

- Hardware architecture of processor and memory
  - Assembly instructions, processors and accelerators, memory hierarchy, storage
- Operating system
  - Processes and threads, scheduling, virtual memory, file systems
- Parallel computing
  - Multi-threading, concurrency, locks
- Networking
  - Socket interface, client-server model, remote procedure calls
- Database and transactions
- Distributed systems and cloud
- *And* practical skills and tools!

# Why You Should Know About Systems

- To make the best use of computer systems
  - Write high-performance software, choose right configurations, use appropriate system solutions to realize your cool ideas, …
- To become a real computer science expert
  - Understand internals, explain behaviors, apply principles, make improvements, …
- To build your own systems
  - DIY hardware/software (e.g., with Raspberry Pi like our labs), push forward computer architectures, invent next-generation computers (e.g., quantum), …

*For all of you from CS, AI, and QI tracks*

# Systems Are Also Fun!

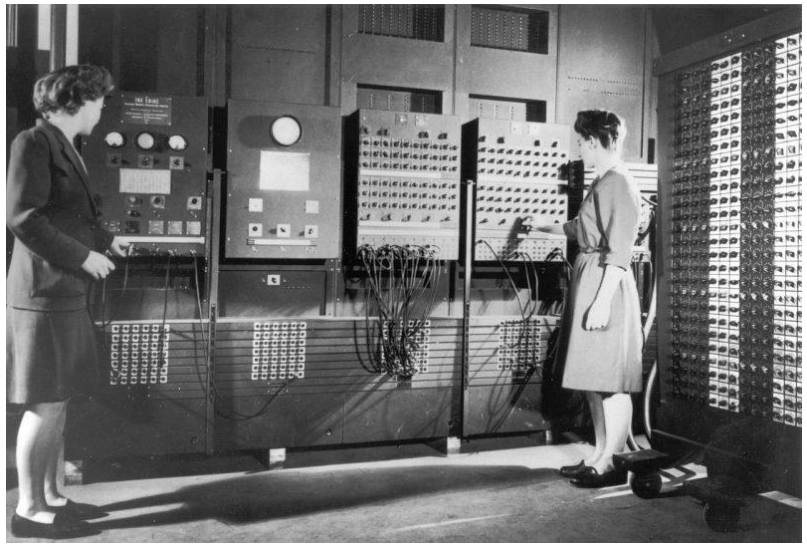
- Many innovations in computer systems
  - To greatly simplify development and usage
  - To drastically improve performance and efficiency
  - To smartly realize ideas and make “impossible” things possible

- Examples: some great ideas in computer systems
  - Abstraction
  - Parallelism
  - Locality
  - Redundancy
  - …
- There are more to come in this course! Hope you enjoy!

# Great Idea 1: Abstraction

- Machines only store and recognize bits
  - Why are we *NOT* programming in binaries today?
- I can already program in C/C++ and/or Python
  - But how is it possible as I know very little about *how* these programs execute?
- In early days (e.g., ENIAC), programmers studied machine blueprints and used external switches to physically re-wired them

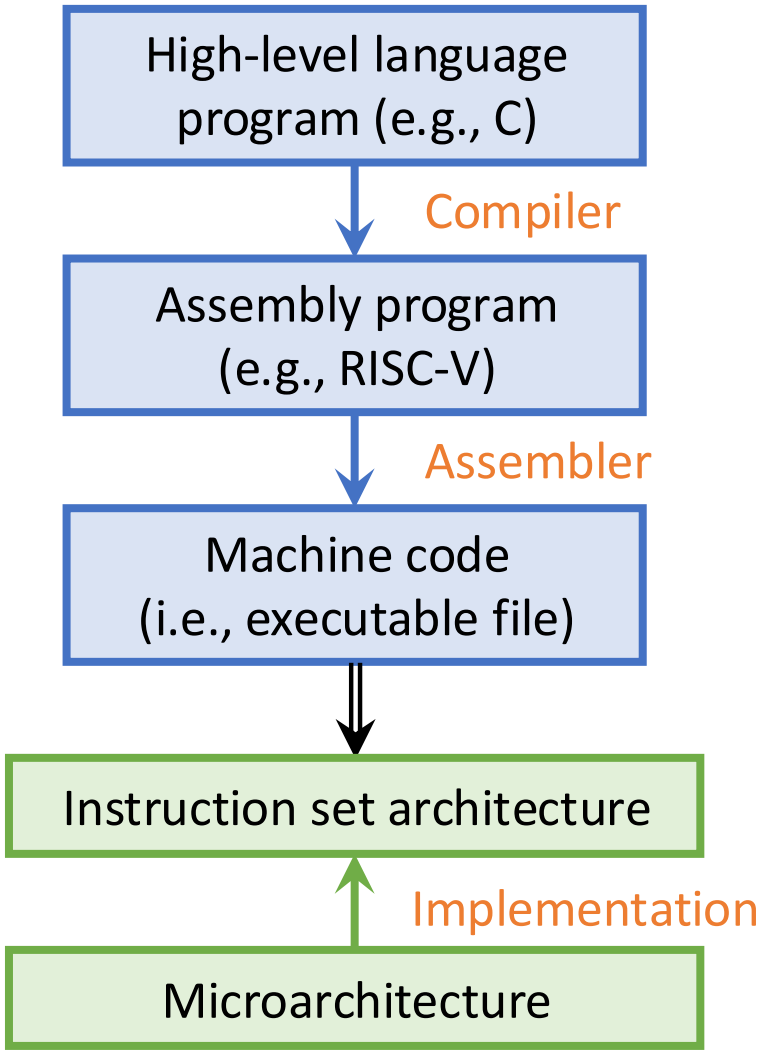


https://www.bricsys.com/blog/computer-programing-a-brief-history

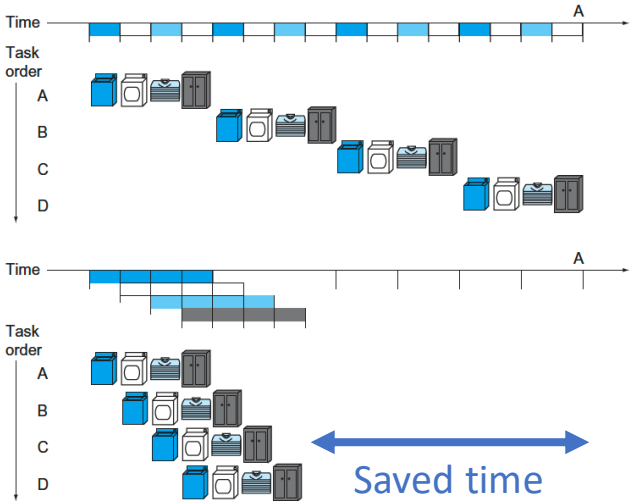
# Great Idea 1: Abstraction

- Various levels of representation/interpretation
  - High-level languages enable programming in human-friendly ways
  - Compiler tools automatically convert programs into machine-readable formats
- Well-defined hardware-software interface
  - I.e., ISA (instruction set architecture)
  - Hide low-level implementation details, i.e., microarchitecture
- The caveat: for functionality, not for performance
  - Aiming for high performance/efficiency usually requires us to break through abstraction levels



# Great Idea 2: Parallelism

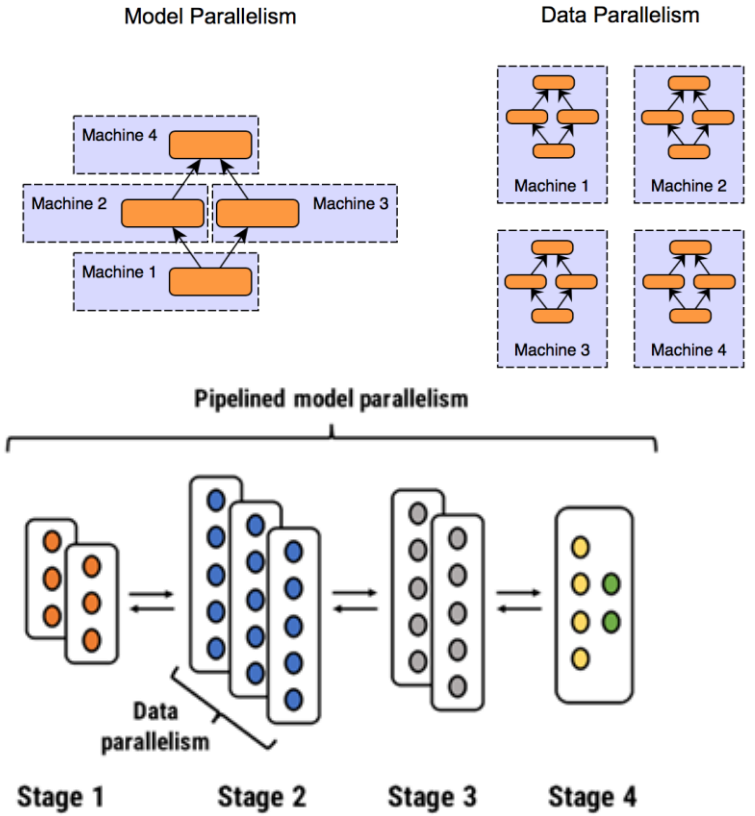
- Parallelism is *everywhere* in systems, at all layers and in various forms
  - Pipelining: overlap different tasks
  - Multi-core: run multiple tasks concurrently
    - Even your small Raspberry Pi is quad-core!
  - Data & model parallelism in neural networks




Pipelining




Multi-core in GPU



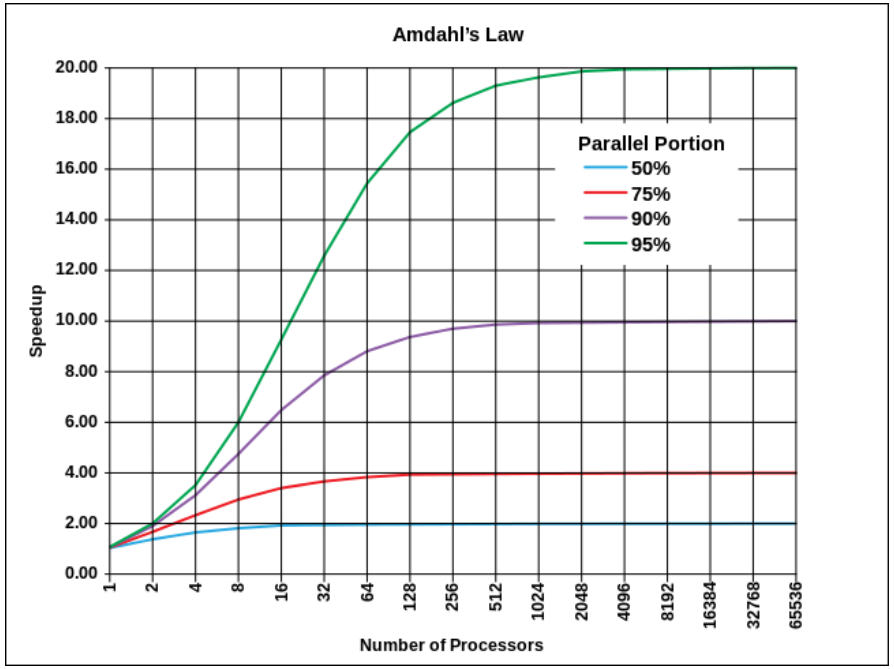

Parallelism in Neural Nets

# Great Idea 2: Parallelism

- The caveat: Amdahl's Law
  - If only a fraction $f$ of a program can be parallelized while the rest is serial, then with $S$ parallel units, the overall speedup would be

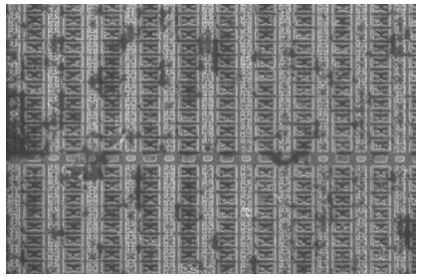
$$\text{Speedup} = \frac{1}{1 - f + \frac{f}{S}}$$

  - Make common cases fast
    - As $f \to 1$, speedup $\to S$
  - But do not over-optimize common cases
    - As $S \to \infty$, speedup $\to \frac{1}{1-f}$
    - Uncommon cases will eventually become common (dominant) cases



# Great Idea 3: Locality

- To store data, we use memory in the system
- Good memories should be both large and fast
- Tradeoff: large memory capacity vs. fast memory access speed
  - Physical laws tell us, large memories are slow, and fast memories are small
    - Simply speaking, traveling a farther distance in a large memory need to take longer time
    - Denser (more bits per area) memory requires heavier (thus slower) phenomena to access
  - No memory technology provides both large capacity and fast speed!



SRAM
~ 1 ns, ~ 1 MB



DRAM
~ 50 ns, ~ 10 GB



SSD
~ 10 µs, ~ 1 TB



Hard Disk
~ 1 ms, ~ 10 TB



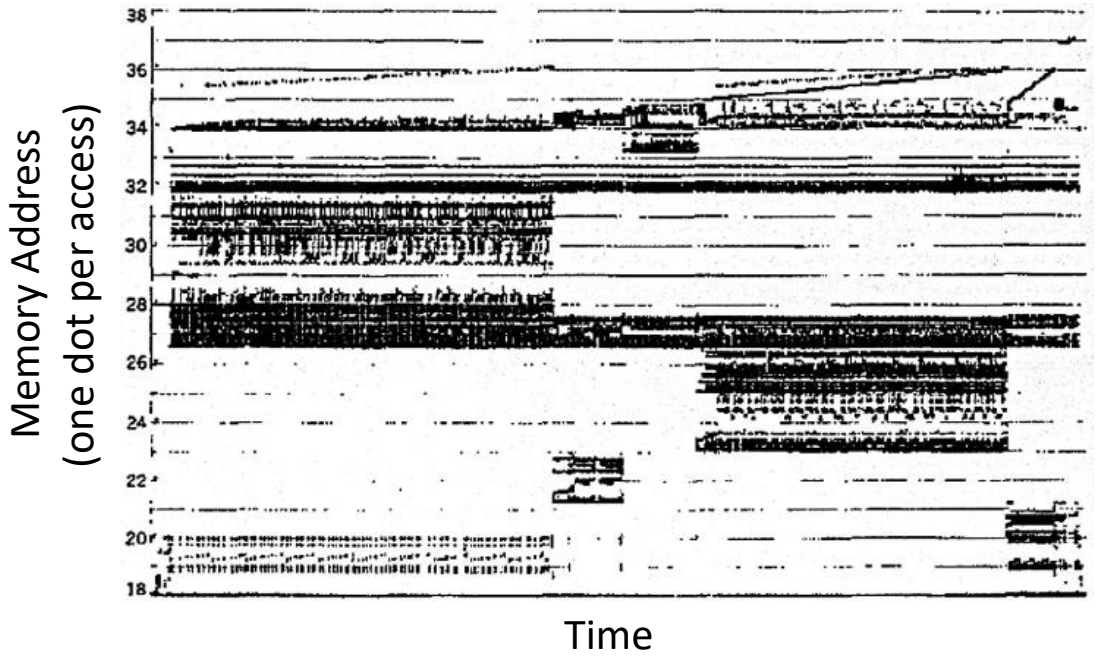
NAS
~ 10 ms, ~ 100 TB

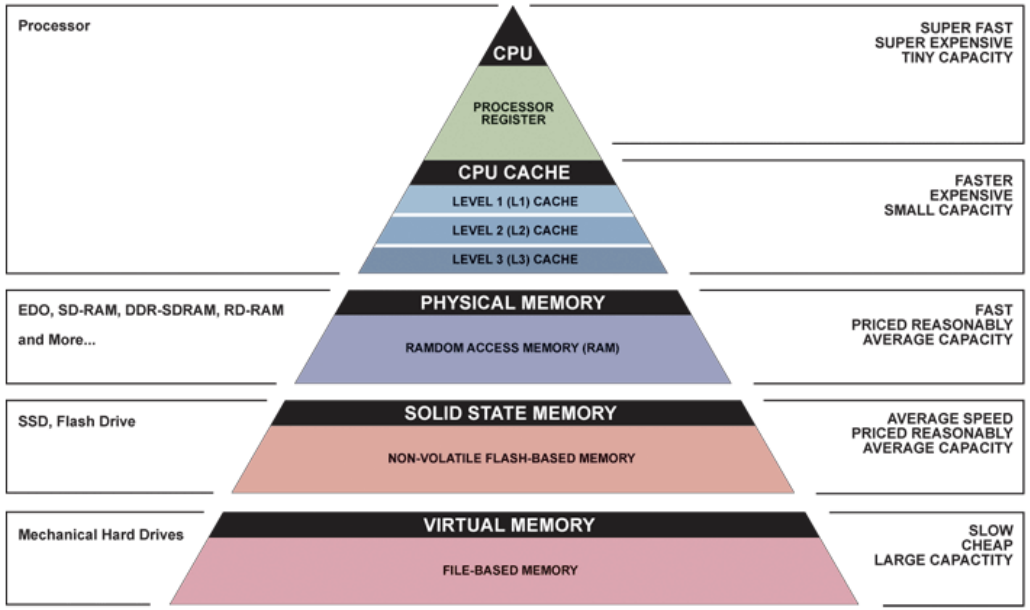
*Faster*
*Smaller*

*Larger*
*Slower*

# Great Idea 3: Locality

- System-level innovations allow us to overcome technology limitations, and present a memory system with both large capacity and fast speed
- Locality: programs work on a relatively small portion of data at any time
- A memory hierarchy exploits locality by caching selective data




Hatfield, Gerald. *Program Restructuring for Virtual Memory*. IBM Systems Journal 10(3): 168-192 (1971)

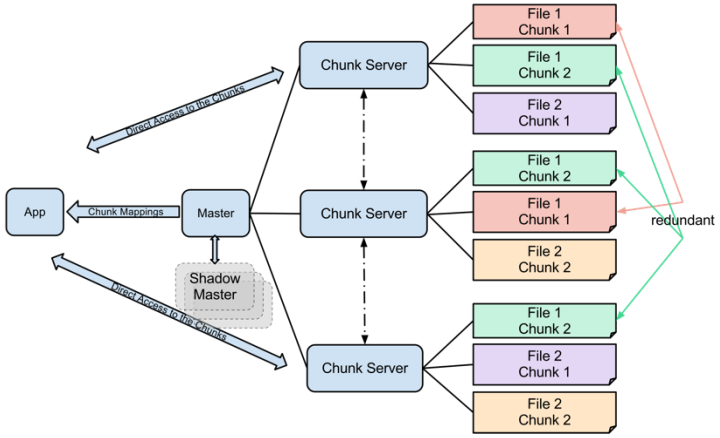
# Great Idea 4: Redundancy

- No system is perfect; there are constantly noise and failures
  - Especially with large scales, e.g., distributed systems and cloud
  - Especially with new technologies, e.g., quantum, persistent memory

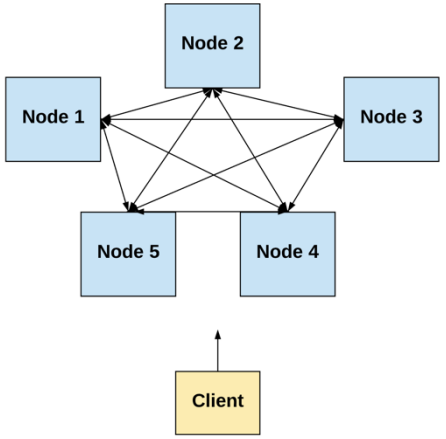
- Example
  - 50,000 servers, 4 disks/server; 4% annual failure rate of each disk
  - On average, you can expert disk failure occurring every hour!
    - 50k * 4 * 4% = 8000 failures per year → ~ 1 failure per hour

# Great Idea 4: Redundancy

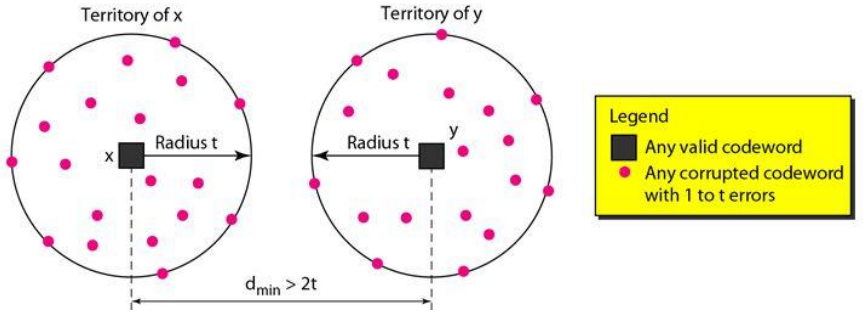
- Improve dependability via redundancy
- Some examples
  - Replicate data multiple times, e.g., Google File System (GFS)
  - Consensus protocols in distributed systems, e.g., Paxos
  - Error correction code, e.g., Hamming code
- Q: How would redundancy affect performance?




Google File System




Distributed Consensus




Error Correction Code

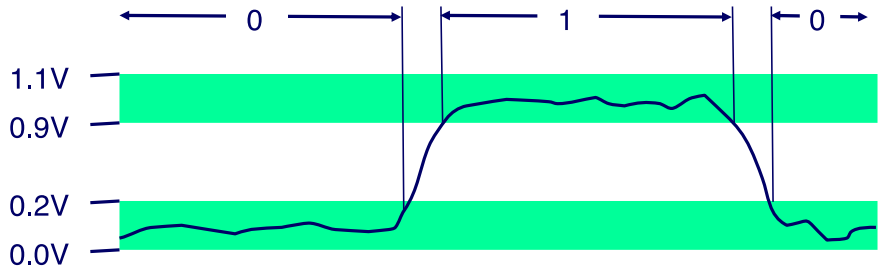
# Data Representation

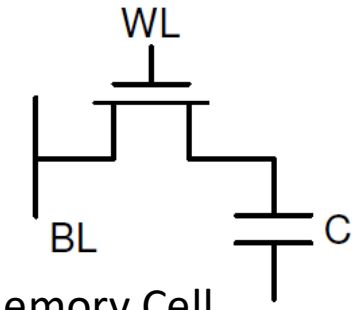
# Outline

- Number representation
  - Integers
  - Floating-point numbers
- Bit manipulation
  - Bitwise operations
  - Addition
  - Multiplication
- Data layout in memory
  - Pointers
  - Arrays
  - Structures and unions
  - Endian

# Everything is Just Bits

- Computer systems are digital circuits
- Digital circuits work on bits: either 0 or 1 (in contrast to analog circuits)
  - Digital circuits are more robust to noise due to quantization
  - Bits are easy to store, e.g., high vs. low voltage




A DRAM Memory Cell

- These only two values are extremely powerful: Boolean algebra
  - By encoding/interpreting bits in various ways
    - E.g., as integers, real numbers, characters, strings, … or even instructions!
  - By operating on bits through arithmetic/logical operations
    - E.g., AND, OR, NOT, XOR, ADD, MULT, …

# Number Representations

- With a certain base $B$, a number $a_{n-1}a_{n-2}\cdots a_2a_1a_0$ has value of

$$v = \sum_{i=0}^{n-1} a_i \times B^i$$

- Examples
  - Decimal (base 10): $123_{10} = 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0$
  - Binary (base 2): $11010011_2 = 2^7 + 2^6 + 2^4 + 2^1 + 2^0 = 211_{10}$
    - Digits: 0 and 1
  - Octal (base 8): $123_8 = 1 \times 8^2 + 2 \times 8^1 + 3 \times 8^0 = 83_{10}$
    - Digits: 0, 1, 2, …, 6, 7
  - Hexadecimal (base 16): $0\text{xFFF} = 15 \times 16^2 + 15 \times 16^1 + 15 \times 16^0 = 4095_{10}$
    - Digits: 0, 1, 2, …, 8, 9, A, B, C, D, E, F
- A useful approximation: $2^{10} \approx 10^3$, so $2^{10} \approx 1\text{k}$, $2^{20} \approx 1\text{M}$, $2^{30} \approx 1\text{G}$

# Encoding Integers As Bits

- Unsigned vs. signed (2's complement)

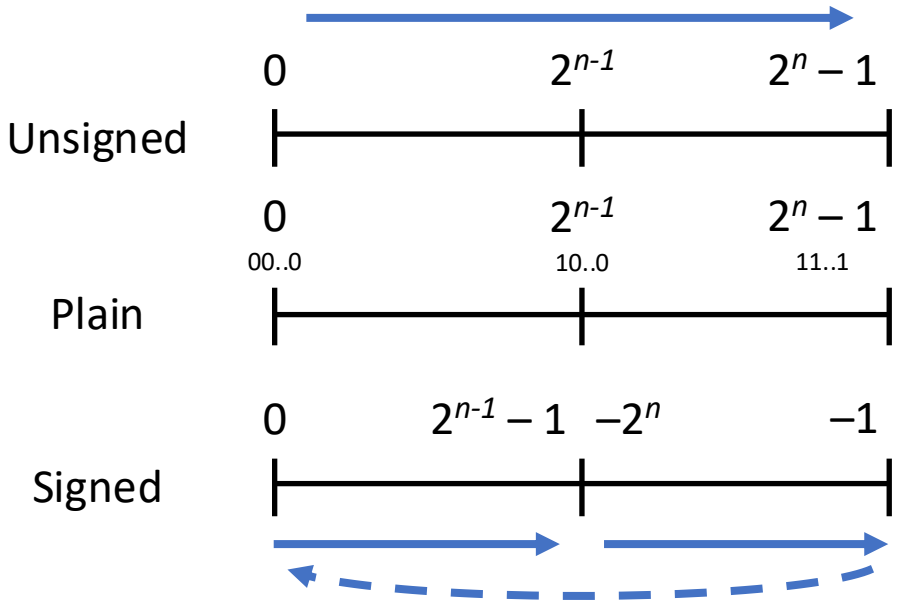
$$v^U = \sum_{i=0}^{n-1} a_i \times 2^i, \qquad v^S = -a_{n-1} \times 2^{n-1} + \sum_{i=0}^{n-2} a_i \times 2^i$$

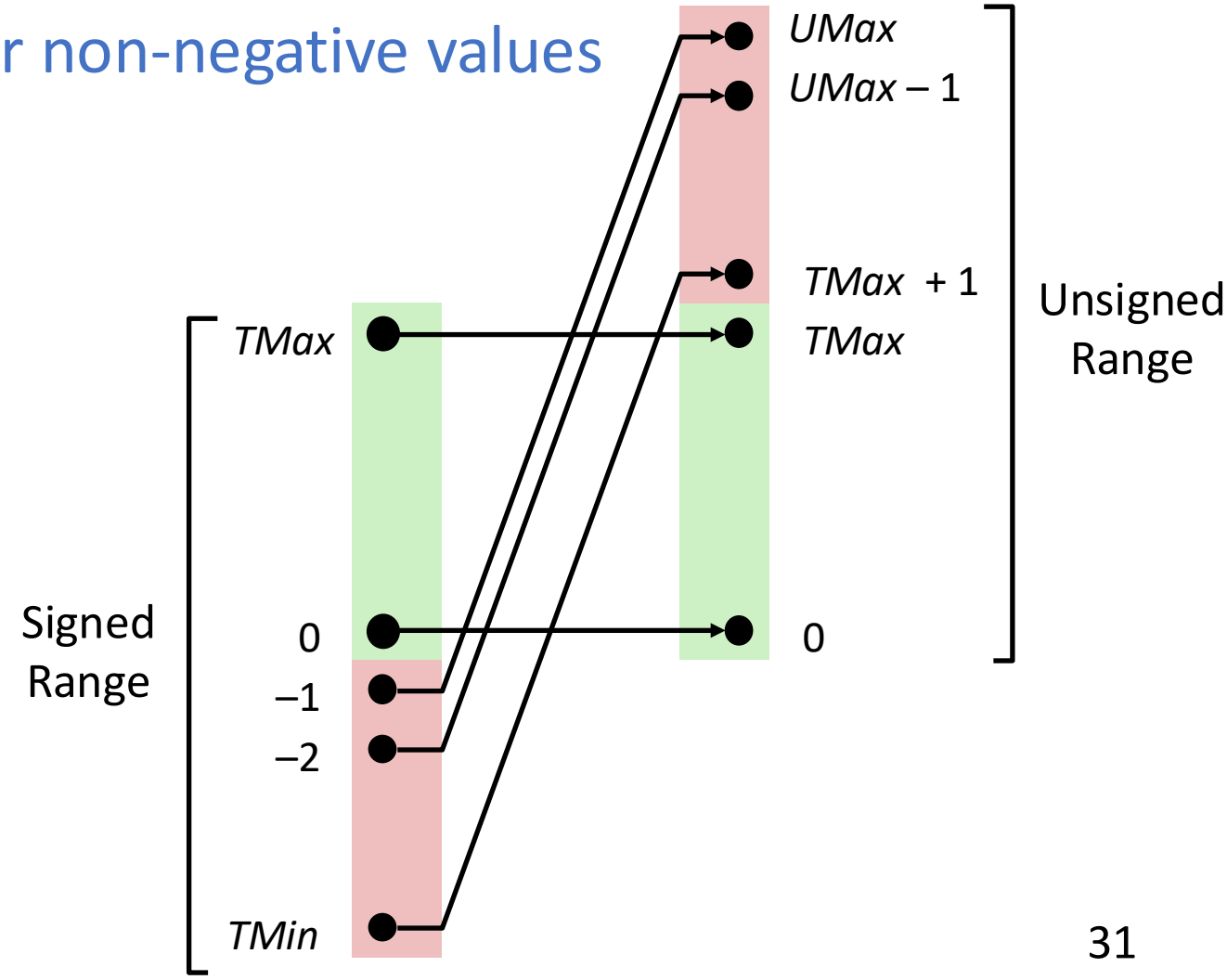
  - E.g., $1001_2$ is 9 as unsigned, or –7 as signed
  - E.g., $1\ldots1_2$ is the max value as unsigned, or –1 as signed

- Get 2's complement of a negative integer
  - Add $2^n$, i.e., subtract its absolute value from $2^n$
  - Or equivalently, invert all bits of its absolute value, and then add 1
    - `~x + 1 == -x`

# Numeric Ranges

- $n$-bit unsigned: 0 to $2^n - 1$; $n$-bit signed: $-2^{n-1}$ to $2^{n-1} - 1$
- Equivalence: same encodings for non-negative values



# Sign Bit and Zero/Sign Extension

- In 2's complement, the most significant bit (MSB) is called sign bit
  - 0 for non-negative, 1 for negative

- Zero-extension for unsigned integers: pad 0's in the front
  - $10110_2$ == $\underline{000}10110_2$
- Sign-extension for signed integers: pad with the sign bit in the front
  - $00110_2$ == $\underline{000}00110_2$, $10110_2$ == $\underline{111}10110_2$
  - Check the correctness by yourself

- The reverse: truncating
  - MSBs are truncated
  - Unsigned: mod operation; signed: similar to mod (but may overflow)

# Signed/Unsigned Casting Surprises

- In C/C++, if there is a mix of unsigned and signed in a single expression (e.g., comparisons, ==, <, >), signed values implicitly cast to unsigned

| Value 1 | Value 2 | Relation | Cast to |
|---|---|---|---|
| 0 | 0U | == | unsigned |
| -1 | 0 | < | signed |
| -1 | 0U | > | unsigned |
| 2147483647 | -2147483648 | > | signed |
| 2147483647U | -2147483648 | < | unsigned |
| -1 | -2 | > | signed |
| (unsigned) -1 | -2 | > | unsigned |
| 2147483647 | 2147483648U | < | unsigned |
| 2147483647 | (int) 2147483648U | > | signed |

Assume 32-bit machine, i.e., range of signed integers: -2,147,483,648 to 2,147,483,647

# When to Use Signed/Unsigned

- Typical bugs when using unsigned

```
unsigned i;
for (i = 100; i >= 0; i--)
  a[i] += a[i+1];
```

- Use unsigned when performing modular arithmetics, multi-precision arithmetics, etc.
  - E.g., in cryptography, when supporting very wide integers
- Use unsigned when manipulating bit strings rather than numbers
  - E.g., using bits to represent sets

# Floating-Point

- IEEE Standard 754
  - Nice for numerical concerns (rounding, ordering, overflow, underflow)
  - Hard for hardware implementation: floating-point ops are expensive in hardware!
  - There exist other standards for floating-point formats
- Encoding: $v = (-1)^S \times M \times 2^E$




  - MSB `s` is sign bit for $S$
  - `exp` field encodes (but not equal to) exponent $E$
  - `frac` field encodes (but not equal to) significand $M$
- Examples in C/C++
  - Single-precision (`float`): 32 bits = 1-bit `s` + 8-bit `exp` + 23-bit `frac`
  - Double-precision (`double`): 64 bits = 1-bit `s` + 11-bit `exp` + 52-bit `frac`
- Normalized, denormalized (exp all 0's), and special (exp all 1's) values

# Typical Data Size in C

- Machine word size: a nominal size of integers and pointers
  - E.g., 32-bit, 64-bit
- Note: most data sizes are implementation-dependent, i.e., depend on processor architectures and compilers

| C Data Type | Typical 32-bit | Typical 64-bit | x86-64 |
|---|---|---|---|
| `char` | 1 | 1 | 1 |
| `short` | 2 | 2 | 2 |
| `int` | 4 | 4 | 4 |
| `long` | 4 | 8 | 8 |
| `float` | 4 | 4 | 4 |
| `double` | 8 | 8 | 8 |
| Pointers | 4 | 8 | 8 |

Unit: Bytes

# Outline

- Number representation
  - Integers
  - Floating-point numbers
- Bit manipulation
  - Bitwise operations
  - Addition
  - Multiplication
- Data layout in memory
  - Pointers
  - Arrays
  - Structures and unions
  - Endian

# Manipulating Bits: Boolean Algebra

- Logical operations

AND, A & B

| & | 0 | 1 |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 1 |

OR, A | B

| \| | 0 | 1 |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 1 |

NOT, ~A

| ~ | |
|---|---|
| 0 | 1 |
| 1 | 0 |

XOR, A ^ B

| ^ | 0 | 1 |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 0 |

- Bitwise operations
  - In C: &, |, ~, ^
  - Apply to "integral" data types: `long`, `int`, `short`, `char`, `unsigned`, …
  - Contrast: `&&`, `||`, `!` in C

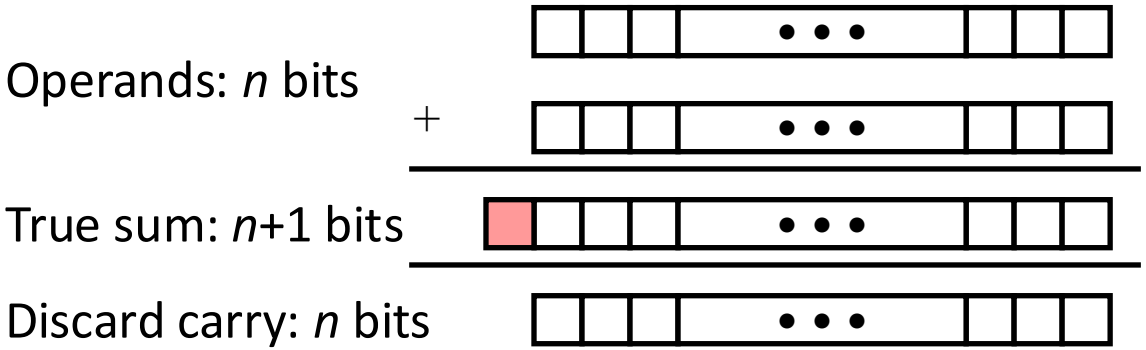
```
  01101001      01101001      01101001
& 01010101    | 01010101    ^ 01010101    ~ 01010101
  --------      --------      --------      --------
  01000001      01111101      00111100      10101010
```

- Shift operations
  - Left shift
  - Logical right shift: zero-extend
  - Arithmetic right shift: sign-extend

| x | 10100010 |
|---|---|
| << 3 | 00010000 |
| Log. >> 2 | 00101000 |
| Arith. >> 2 | 11101000 |

# Addition

- Just as decimal addition rules
  - Carry bit goes to the next bit
  - The final carry bit is discarded




- Same function for unsigned & signed

```
int s, t, u, v;
s = (int)((unsigned)u + (unsigned)v);
t = u + v
// s == t
```

  - But only when no overflow happens!

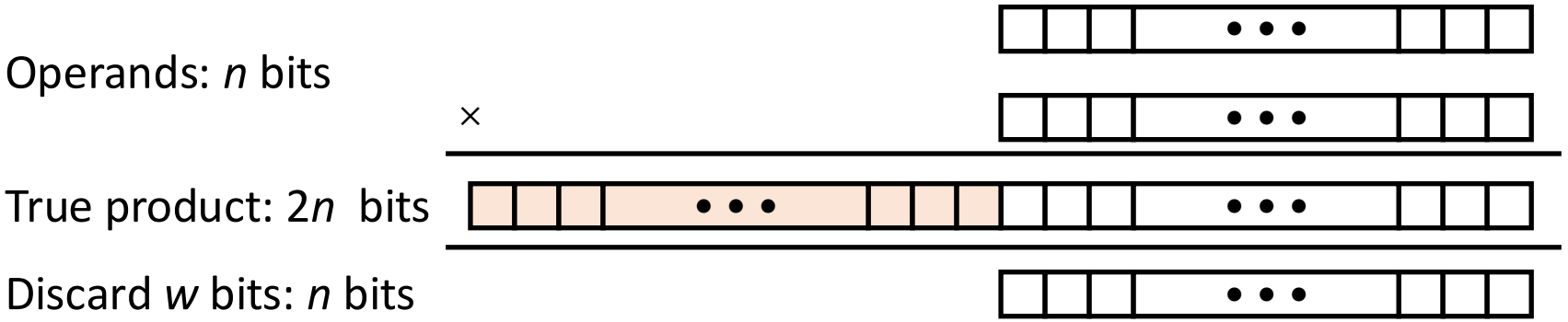
```
  0011   3      0011   3      1101  −3
+ 0010   2    + 1110  −2    + 1110  −2
------        ------        ------
  0101   5     10001   1     11011  −5
```

- Overflow: when result cannot fit in $n$ bits
  - The MSB (i.e., the final carry) is thrown away
  - Overflow occurs when throwing away this MSB cause change of sign bit (see more in Appendix)

```
 00111   7      1000  −8
+ 00001   1    + 1111  −1
------        ------
 01000  −8     10111   7
Overflow!     Overflow!
```

# Unsigned Multiplication

- Shift-and-add
- Up to 2*n*-bit results
  - Discard higher bits




- There exist faster algorithms, e.g., Booth encoding

| | Unsigned | | Signed |
|---|---|---|---|
| | 0011 | 3 | 3 |
| × | 1110 | 14 | –2 |
| | 0011 | | |
| | 0011 | | |
| | 0011 | | |
| | 0101010 | 42 | |
| | 1010 | | –6 |

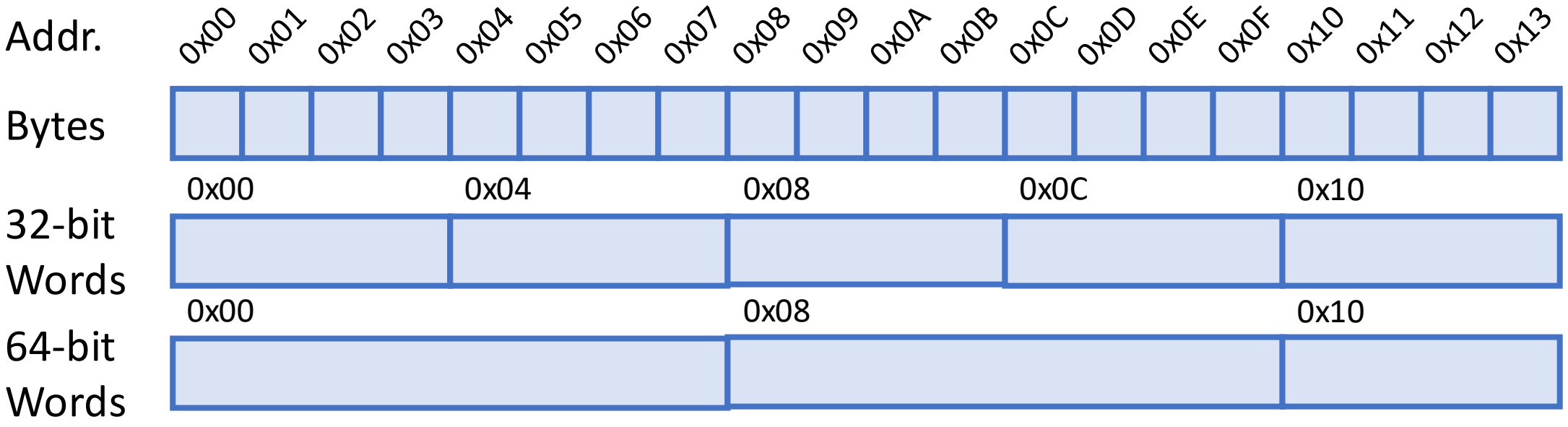
- Signed multiplication has slightly different function
  - Lower bits are the same
- Power-of-2 multiplication is left shift
  - $u << k$ gives $u \times 2^k$
  - Compiler can automatically generate shift-and-add for multiplying by constant

# Outline

- Number representation
  - Integers
  - Floating-point numbers
- Bit manipulation
  - Bitwise operations
  - Addition
  - Multiplication
- Data layout in memory
  - Pointers
  - Arrays
  - Structures and unions
  - Endian

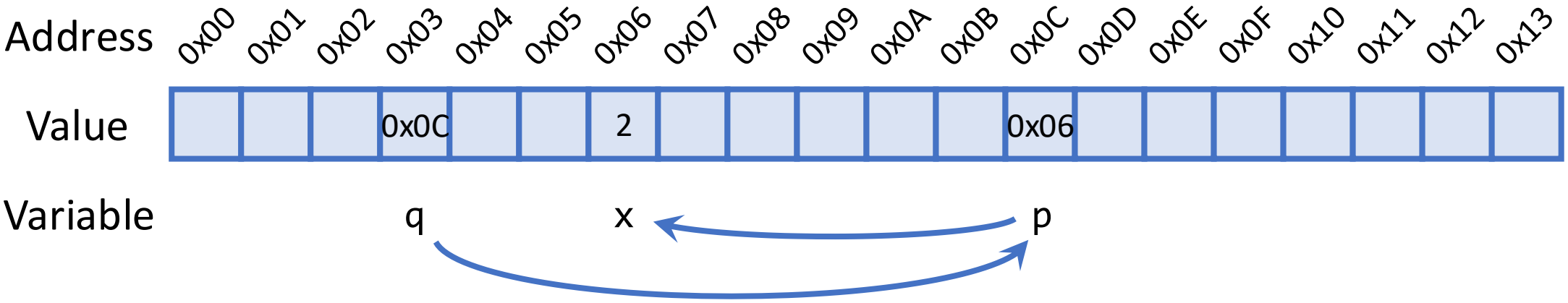
# Byte-Oriented Memory Addressing

- Memory: virtually a very large array of bytes
- Programs refer to data/instructions by address, like array index
  - A pointer variable in C/C++ stores an address
- Addresses specify byte locations, i.e., address of first byte in a word
- Aligned address: multiple of word size



# Pointers in C/C++

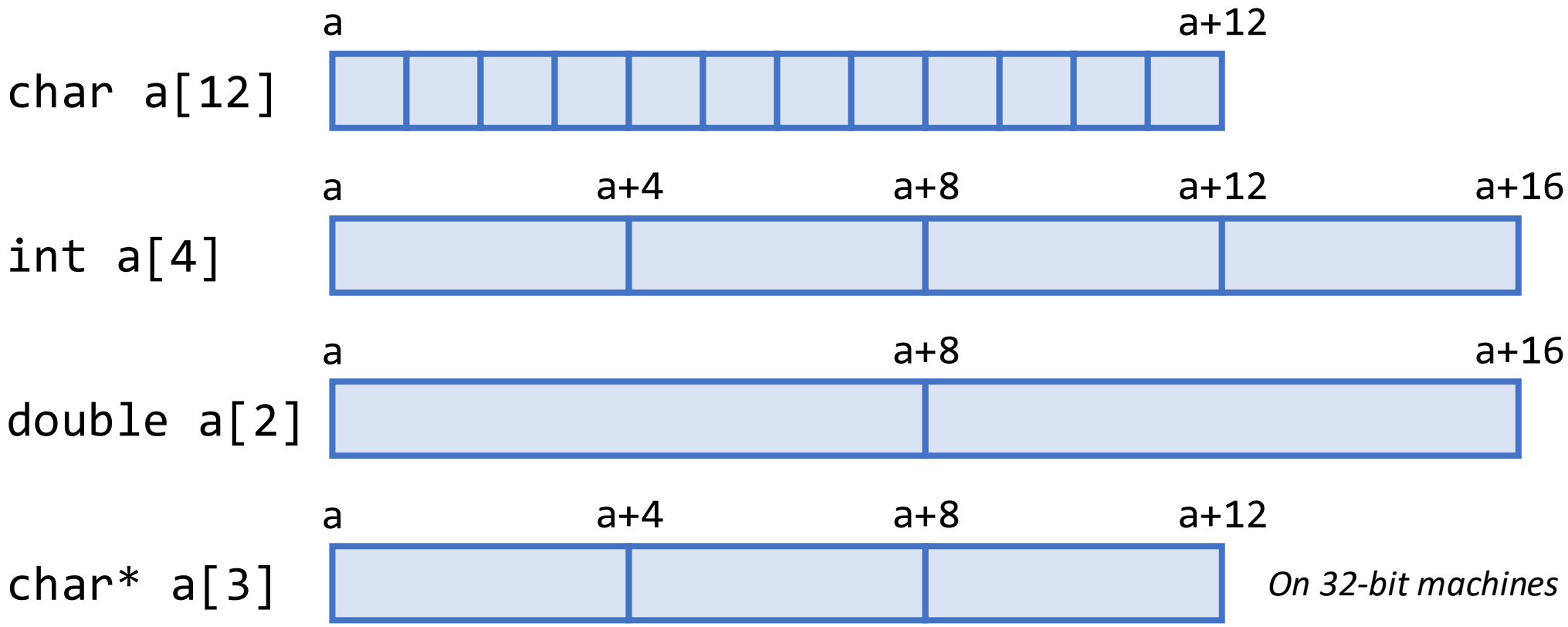
- Pointer: a variable that contains the address of another variable
  - E.g., `int* p = &x;`
  - p is a pointer that contains the address of x, so the value of p is x's address, 0x06
  - p itself is a variable, needs its own memory space and has its own address, 0x0C; do not confuse p's own address vs. the address stored in p
  - We can even have another pointer pointing to p, e.g., `int** q = &p;`




- Address operator & and dereference operator *
  - E.g., `p = &x; x = *p;`

# Arrays

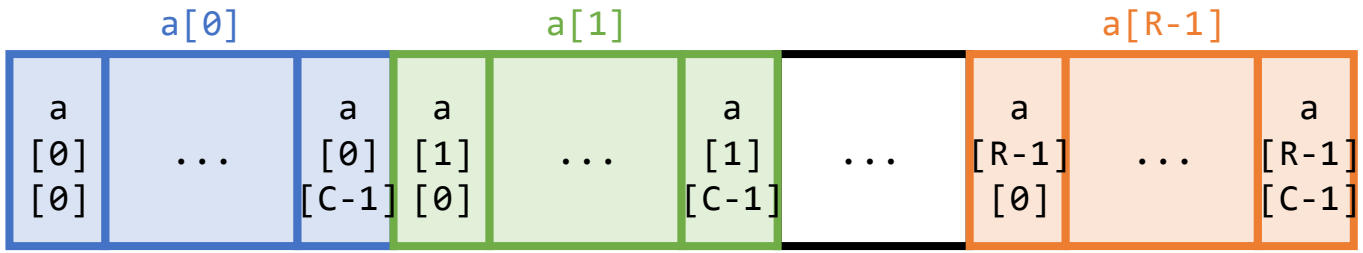
- `Type a[L]` has contiguous memory of `L*sizeof(Type)` bytes
- a can also be used like a pointer, e.g., `a == &a[0]`, `a + 1 == &a[1]`



# Multi-Dimensional Arrays

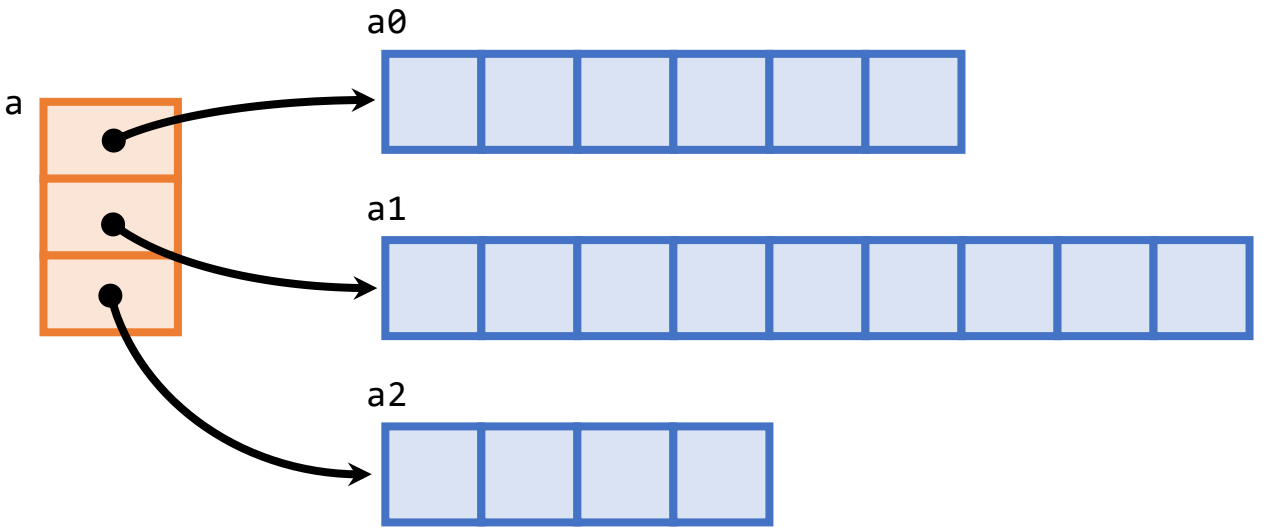
- Declaration: `Type a[R][C]`
  - R rows, C columns, in total `R * C * sizeof(Type)` bytes
- Row-major ordering in C
  - Each row is allocated contiguously, and all rows are allocated contiguously
- Byte offset of `a[i][j]` in array = `(i * C + j) * sizeof(Type)`
  - In pointer arithmetics, `&a[i][j] == a + (i * C + j)`

`int a[R][C]`



# Multi-Level Arrays

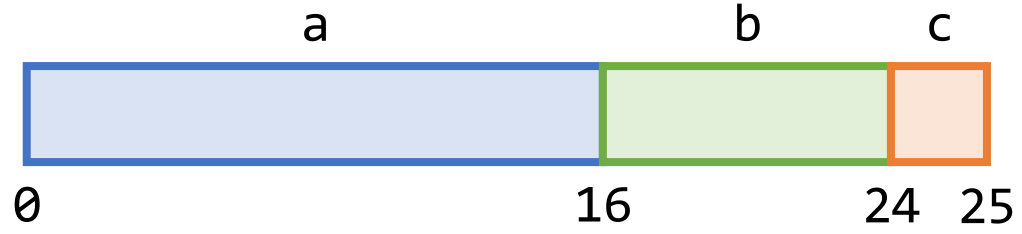
- Declaration
  - `Type a0[N0], a1[N1], ...;`
  - `Type *a[M];`
  - `a[0] = a0; a[1] = a1; ...`



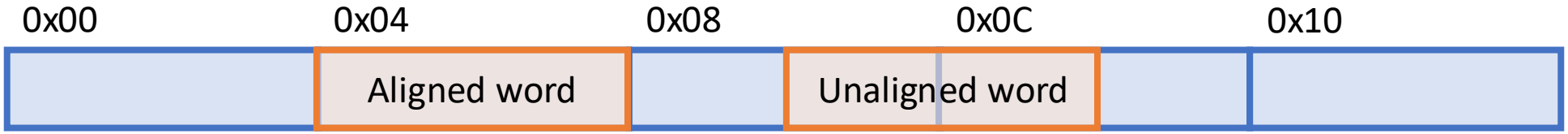

- Each element of array `a` is a pointer, which points to an array
- The second-level arrays are not necessarily contiguous in memory
- Access `a[i][j]`: two memory accesses
  - First get the address of the second-level array: `a[i]`
  - Then access the element within the array: `a[i][j]`
- Syntax similar to multi-dimensional arrays, but accesses are different

# Structure

```
struct foo {
  int a[4];
  double b;
  char c;
} x;
```



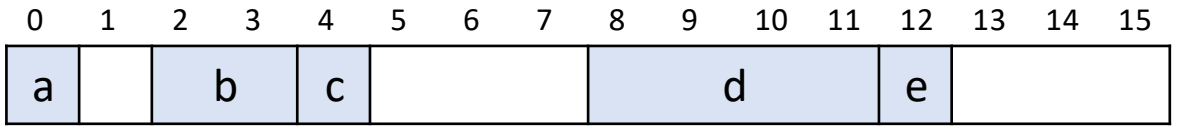

- Structure uses a block of memory to hold all its fields
- Fields are guaranteed to be ordered, according to declaration
- Overall size and offset of each field are determined by compiler and ISA
  - So `sizeof(x.a) + sizeof(x.b)` often does not give offset of `x.c`
- Alignment: data fall on addresses that are multiples of the data size
  - Some ISAs require data alignment
  - Other ISAs do not require data alignment, so it is up to the compiler



# Alignment for Structures

- Satisfy alignment in structures with padding
  - Within structure: each field satisfies its own alignment requirement
  - Overall structure: align to the largest alignment requirement of all fields

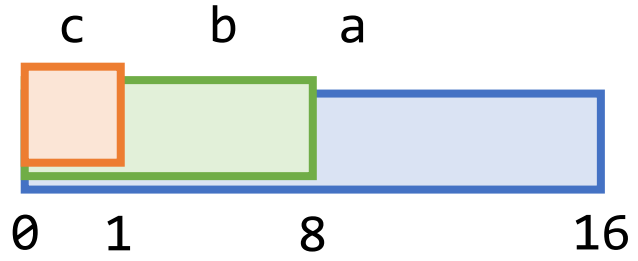
```
struct foo {   // starts at 0x1000 , must be 4-byte aligned
  char a;      // 1-byte, starts at 0x1000
  short b;     // 2-byte, starts at 0x1002
  char c;      // 1-byte, starts at 0x1004
  int d;       // 4-byte, starts at 0x1008
  char e;      // 1-byte, starts at 0x100c
}              // total 16 bytes
```




- To save space, put large-size fields first
  - E.g., with the order of `d, b, a, c, e`, the total size is 9 bytes
  - But this may decrease code readability

# Union

```
union foo {
  int a[4];
  double b;
  char c;
} x;
```

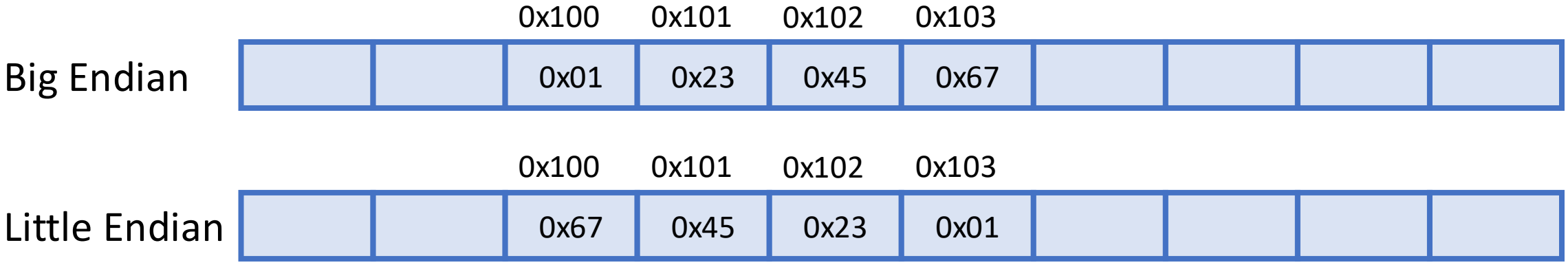



- Union uses a block of memory to hold the largest of its fields
- Fields are overlapped and share memory space

- Q: If we access `c`, which byte of `a` would we get?
  - MSB or LSB? Of `a[0]` or `a[3]`?

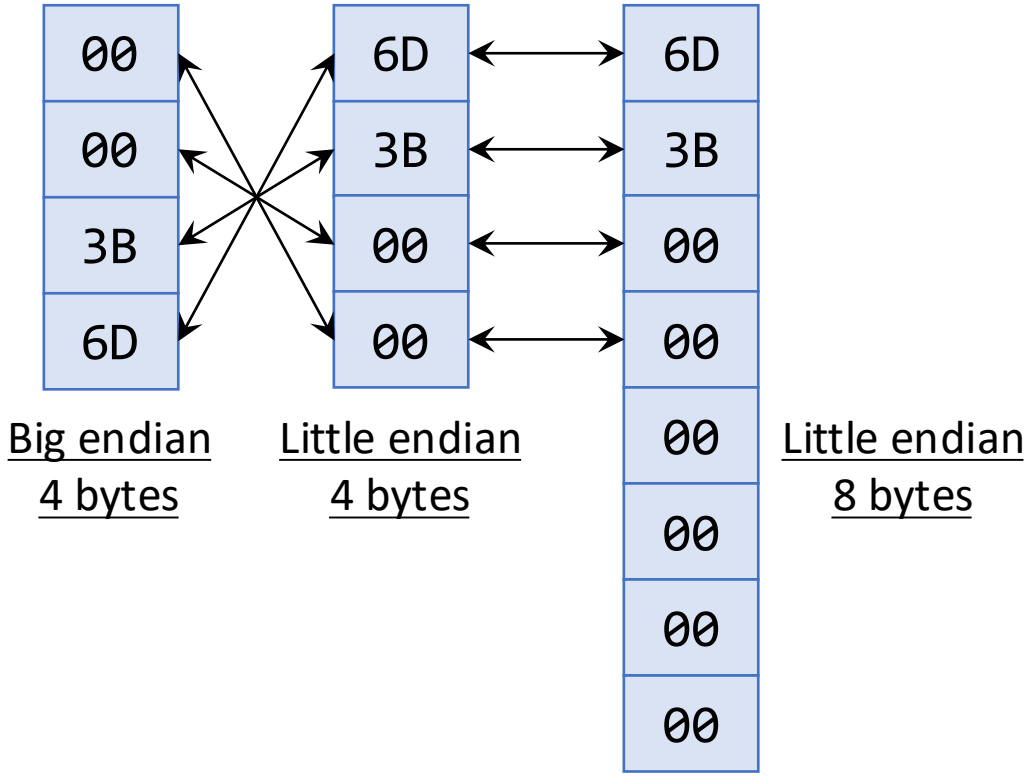
# Byte Ordering in A Word

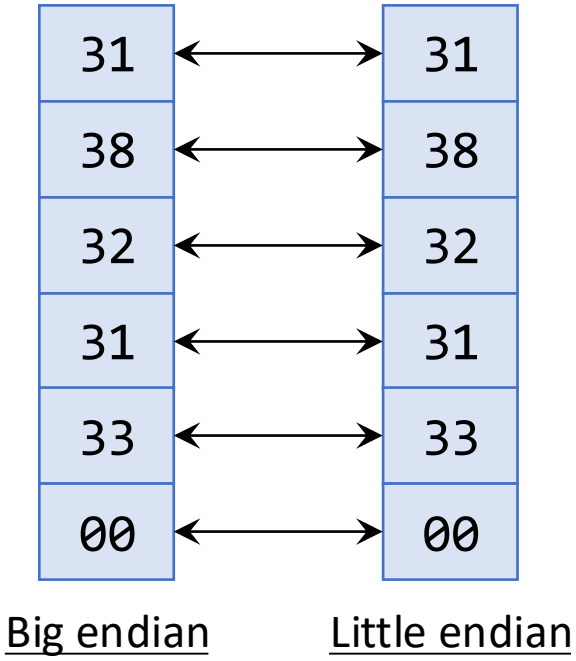
- A word contains multiple bytes; how are these bytes ordered?
  - Matters when using union types, when transferring data between systems, …
- Big endian: least significant byte has highest address
  - E.g., MIPS, Internet (i.e., network order)
- Little endian: least significant byte has lowest address
  - E.g., x86, ARM, RISC-V
- Example: a 4-byte word with value 0x01234567 and address 0x100



# Data Layouts In Memory

- Integers, e.g., `long int C = 15213;` i.e., 0x3B6D
- Strings, e.g., `char S[6] = "18213";`
  - Always as a array of chars, not affected by byte ordering

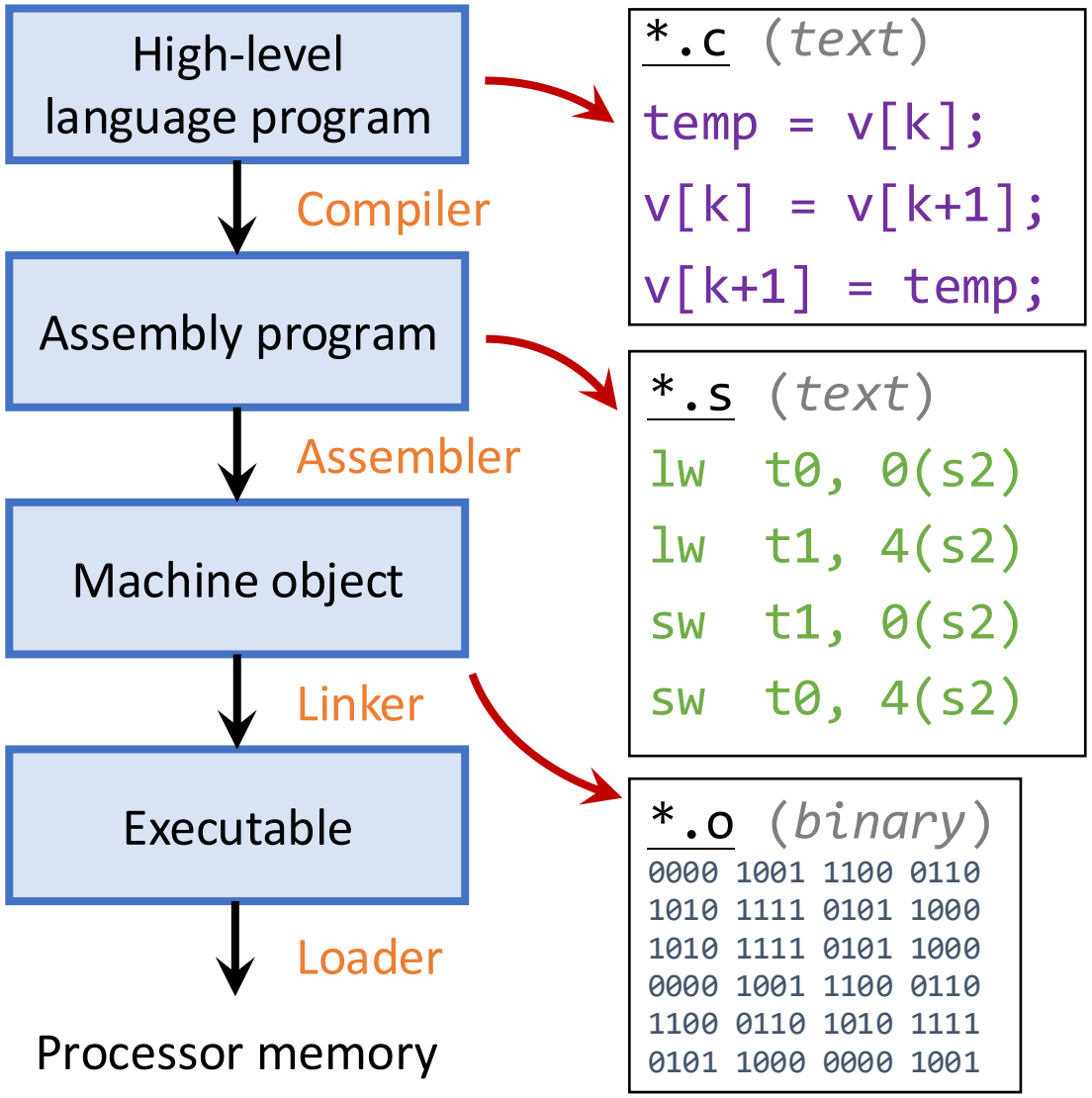


# Program Representation

Recall: everything is just bits; so is a program

# Code Compilation and Translation




- A program is compiled into a sequence of instructions
  - Instructions are encoded as bits
- Machine code: the binary sequence that encodes program instructions
  - Actual 0's and 1's stored in the memory
- Assembly code: the text representation of machine code
  - Human-readable instruction sequence
- Disassembler: `objdump -d foo`

# Compilation vs. Interpretation

- Compilation: programs are converted to machine code, which is then directly executed on hardware
  - E.g., C/C++
  - Machine code is system-dependent, e.g., x86, ARM, etc.

- Interpretation: an interpreter parses programs and directly executes each statement, without first compiling into machine code
  - E.g., Java, Python
  - Usually the text program is first converted into system-independent bytecode, and the interpreter parses and executes the bytecode
  - An optimization – just-in-time (JIT) compilation: bytecode can be compiled into machine code on the fly at runtime, so it can directly execute (faster) rather than being interpreted (slower)

# Compilation vs. Interpretation Comparison

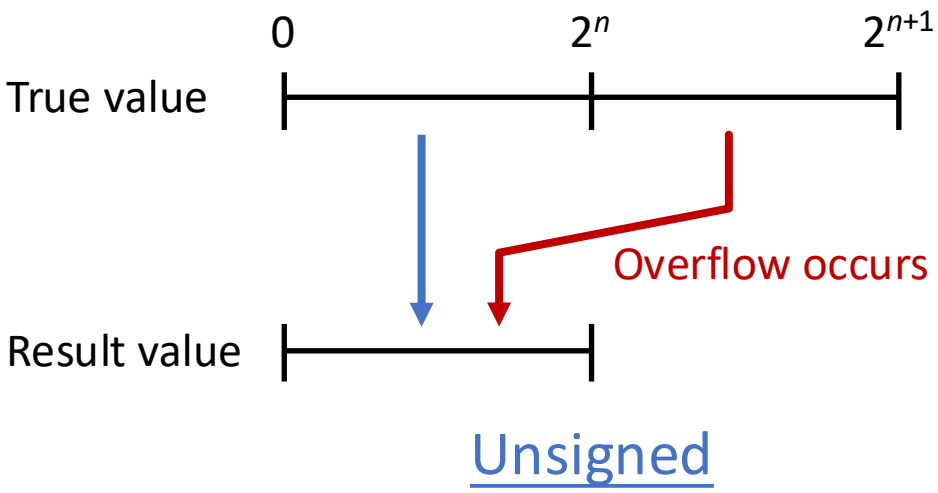
- Compilation offers excellent runtime performance
  - But these days, most performance is in libraries; e.g., Python can invoke numpy, PyTorch can invoke cudnn, both actually implemented and compiled using C/C++
- Compilation does the hard work once; interpretation pays every time
  - But JIT compilation in interpretation help alleviate this issue
- Need recompile every time after code change (usually acceptable, but sometimes cumbersome)
- Compiled files (.o, exe) are processor- and OS-specific, not portable
- Interpreters are easier to implement than compilers
  - Interpreters are closer to higher, human-readable levels

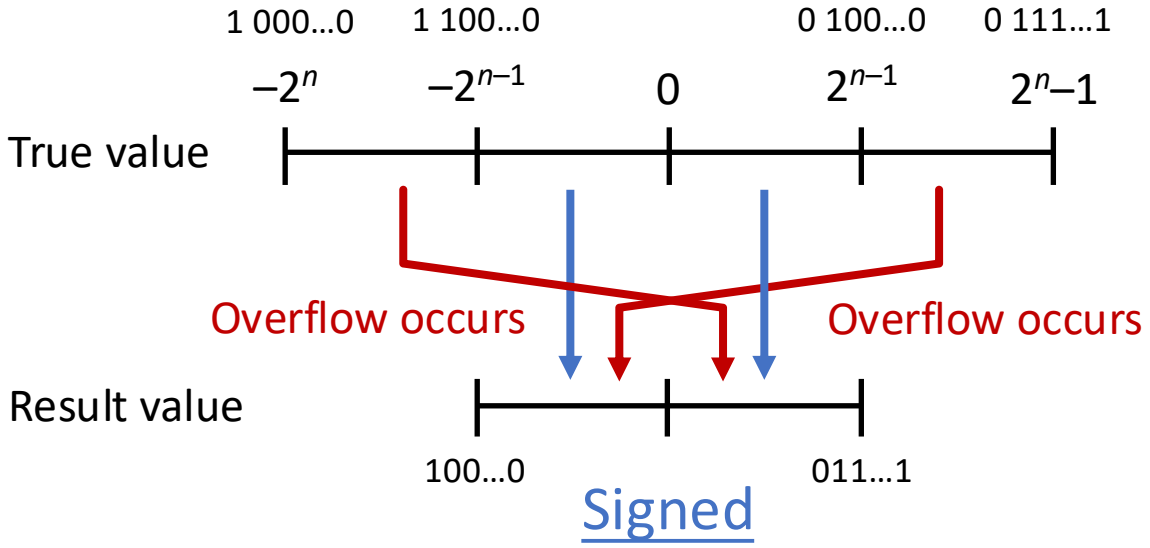
# Next Lecture:
# Assembly and Processors

# Appendix

# Arithmetic Overflow

- Different effects for signed (2's complement) and unsigned numbers when simply throwing away the MSB in the overflow results
  - E.g., $1011_2 + 0111_2$
    - Treat as unsigned: $(0)1011_2 + (0)0111_2 = \cancel{(1)}0010_2$ # 11 + 7 = 2, overflow
    - Treat as signed: $(1)1011_2 + (0)0111_2 = \cancel{(0)}0010_2$ # (–5) + 7 = 2, no overflow



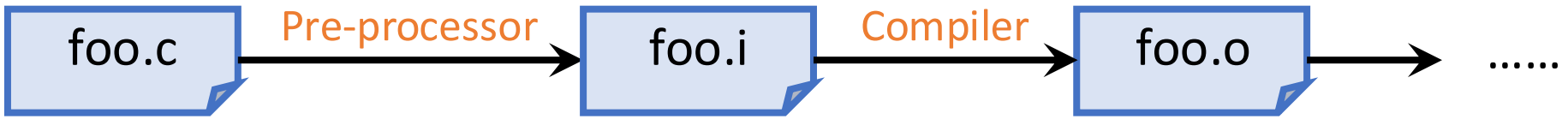



# Outline

- Code compilation
  - Pre-processing
  - Compilation
  - Linking

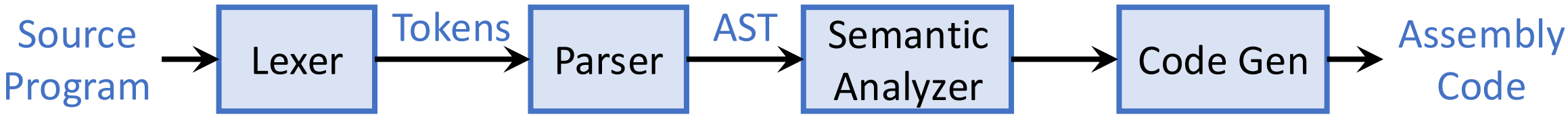
# C Pre-Processing

- Pre-process before actual compilation




- Resolve pre-processing commands (starting with #)
  - `#include “file.h”` /* insert the header */
  - `#include <stdio.h>`
  - `#define M_PI (3.14159)` /* define a constant */
  - `#define MAX(x,y) ((x) > (y) ? (x) : (y))` /* define a “function” */
    - Q: why the parentheses?
  - `#if #endif` /* conditional inclusion */
- Remove comments

# Steps In A Typical Compiler




- Lexer/tokenizer
  - Turn the input program into tokens (operators, operands, function calls, …)
- Parser
  - Turn the tokens into abstract syntax tree (AST), which captures the relationship between tokens (e.g., an operator has two operands)
- Semantic analyzer and optimizer
  - Check for semantic errors; reorganize code to optimize
- Code generation
  - Output assembly code

# Linker

- Combine several object files (`.o`) into a single executable
  - Enable separate compilation of files; only recompile changed files in a big project
  - Enable use of third-party static/dynamic libraries, e.g., `malloc`, `printf`
- Static libs, e.g., `.a` files in Linux, are directly combined into executable
- Some libs are dynamically linked, e.g., `.so` in Linux and DLL in Windows
  - Linking occurs lazily when program begins execution
  - Benefits: avoid duplication across executables; can fix bugs or upgrade libs without recompiling executables; …
- Key task during linking: resolve references between files
  - Symbol resolution: functions & global variables in other files, with symbol tables
  - Relocation: adjust relative locations of functions/data after merging files; store locations that need to be fixed up later (e.g., dynamic libs) in a relocation table